2

Лекция 16–17 февраля

Обходы и деревья

Константин Чухарев

Дискретная математика — Весна 2027

Обходы графа

> *Не все, кто блуждает, потеряны.*
>
> ***— Дж. Р. Р. Толкин***

Зачем обходить граф?

Граф задан, но из пары множеств мало что видно.

Обход превращает граф в структуру с порядком: что за чем посещено.

- Найти компоненты связности.
- Найти кратчайшие пути от старта.
- Найти циклы и топологический порядок.
- Построить остов.

Любой обход честен к графу: каждая вершина и каждое ребро просматриваются $O(1)$ раз, итого $O(V + E)$.

Всюду ниже граф хранится списками смежности: сумма длин всех списков равна $2\,|E|$, и линейность достижима.

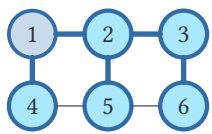
На матрице смежности один только просмотр соседей стоит уже $O(V)$ на вершину.

Поиск в ширину

BFS расходится от старта волной: очередь хранит границу волны, вершины текущего слоя.

1. Помещаем стартовую вершину в очередь.
2. Пока очередь не пуста:
   - извлекаем вершину u из головы;
   - всех непосещённых соседей v помечаем и ставим в хвост.

Очередь честная: кто пришёл раньше, тот выходит раньше.



BFS идёт слоями: соседи старта, затем соседи соседей.

Слой вершины — её расстояние от старта.

Трассировка BFS: шаг за шагом

Пройдём алгоритм по решётке BFS: старт из вершины 1, соседи перебираются по возрастанию номеров.

Шаг	Извлечена	Обнаружены	Очередь после шага
1	—	1	[1]
2	1	2, 4	[2, 4]
3	2	3, 5	[4, 3, 5]
4	4	—	[3, 5]
5	3	6	[5, 6]
6	5	—	[6]
7	6	—	[]

Очередь хранит вершины не более двух соседних слоёв, и слои идут подряд.

Что дала трассировка?

Пример (Итоги прогона)

Порядок извлечения: $1, 2, 4, 3, 5, 6$.

Расстояния: $d(1) = 0$, $d(2) = d(4) = 1$, $d(3) = d(5) = 2$, $d(6) = 3$.

Родители: $\mathrm{parent}(2) = \mathrm{parent}(4) = 1$, дальше $\mathrm{parent}(3) = \mathrm{parent}(5) = 2$ и $\mathrm{parent}(6) = 3$.

Путь до 6 собирается с конца по родительским ссылкам: $1 \to 2 \to 3 \to 6$.

Родительские ссылки образуют дерево кратчайших путей: по ним восстанавливается маршрут до любой вершины.

Расстояние вершины — это номер слоя, поэтому порядок извлечения из очереди уже отсортирован по расстоянию.

Почему BFS идёт кратчайшими путями?

Теорема 1 (Корректность расстояний BFS)

В невзвешенном графе после обхода из s значение $d(v)$ равно длине кратчайшего пути из s в v.

Доказательство

Докажем индукцией по k: каждая вершина слоя k получает $d = k$ и извлекается после всех вершин меньших слоёв.

База. Слой 0 — это старт: он обнаружен и извлечён первым.

Шаг. Вершина v слоя $k+1$ имеет соседа u слоя k — предпоследнюю вершину кратчайшего пути до v. При обработке u белая v получает $d(v) = k+1$: сосед на расстоянии $k' < k$ дал бы $d(v) \le k' + 1 \le k$. Очередь выносит вершины в порядке поступления, поэтому слои выходят подряд.

Слои и кратчайшие пути

В социальной сети слои BFS — круги знакомых: друзья, друзья друзей, друзья друзей друзей.

Пример (Шесть рукопожатий)

Эксперимент Милгрэма (1967): незнакомцам доверяли письма, и каждый участник передавал конверт знакомому, который казался ближе к адресату.

Цепочки дошедших конвертов содержали в среднем около шести передач — слои BFS измеряют те же «рукопожатия» в социальной сети.

Рёбра $s - a$ веса 10, $s - b$ и $b - a$ веса 1: BFS обнаружит a прямо из s и вернёт путь веса 10, хотя путь $s - b - a$ весит 2.

Взвешенный минимум ищет Дейкстра.

Компоненты за один обход

Компоненты связности считаются обходом.

Запуск из непосещённой вершины собирает ровно её компоненту.

Повторяем запуски, пока остаются непосещённые вершины.

Пример (Две компоненты)

Пусть рёбра: $1 - 2$, $2 - 3$, $3 - 4$ и отдельно $5 - 6$, $6 - 7$.

Запуск BFS из 1 даёт порядок $1, 2, 3, 4$ — компонента собрана целиком.

Вершина 5 осталась белой: второй запуск даёт $5, 6, 7$ — итого две компоненты.

Число запусков — число компонент, а общее время остаётся $O(V + E)$: каждая вершина по-прежнему посещена один раз.

Двудольность: раскраска из BFS

BFS красит слои попеременно: старт в цвет A, его соседи в цвет B, следующий слой снова в A.

Пример (Раскраска решётки)

Слои из трассировки: $\{1\}$, затем $\{2, 4\}$, затем $\{3, 5\}$, затем $\{6\}$.

Чётные слои дают долю $A = \{1, 3, 5\}$, нечётные — долю $B = \{2, 4, 6\}$.

Все семь рёбер соединяют разные доли — разбиение получено одним обходом.

Слои без внутренних рёбер — двудольность, а для большего числа цветов простого критерия нет.

Критерий двудольности

Теорема 2 (Критерий двудольности)

Граф двудолен тогда и только тогда, когда в нём нет циклов нечётной длины.

Доказательство

Докажем эквивалентность в обе стороны.

Двудольность запрещает нечётные циклы. Доли вдоль цикла чередуются, и замыкающее ребро $v_k - v_1$ соединяет разные доли: k чётен.

Отсутствие нечётных циклов даёт раскраску. Покрасим компоненты слоями BFS: цвет вершины — чётность её слоя. Ребро $x - y$ одного цвета даёт замкнутый обход длины $d(x) + d(y) + 1$, нечётной из-за равной чётности $d(x)$ и $d(y)$. Самый короткий нечётный замкнутый обход — цикл: повтор вершины разрезал бы его на два замкнутых куска, один из которых снова нечётен, но короче — противоречие.

Пятиугольник: нечётный цикл

Пример (Пятиугольник)

Пусть рёбра: $1-2$, $2-3$, $3-4$, $4-5$, $5-1$.

Слои BFS из 1: $\{1\}$, затем $\{2,5\}$, затем $\{3,4\}$.

Доли по чётности: $A=\{1,3,4\}$ и $B=\{2,5\}$ — а ребро $3-4$ лежит внутри A.

Цикл $1-2-3-4-5-1$ имеет длину 5, и два цвета ему не подходят ни в каком назначении.

Конфликт может обнаружиться не в первом слое: начало раскраски выглядит безупречно, а противоречие ждёт на дальнем слое.

Вывод о двудольности законен только после полной раскраски всей компоненты.

Поиск в глубину

DFS идёт наоборот: первый же непосещённый сосед уводит вглубь, возврат — только в тупике.

1. Для каждой непосещённой вершины u запускаем $\mathrm{DFS}(u)$.
2. $\mathrm{DFS}(u)$:
   - помечает u и записывает время входа $\mathrm{pre}(u)$;
   - для каждого непосещённого соседа v вызывает $\mathrm{DFS}(v)$;
   - записывает время выхода $\mathrm{post}(u)$.

Структура данных — стек вызовов: уходим вглубь, пока можно.

> DFS не ищет ближайшее — он исследует надолго и тщательно.
>
> Времена pre и post превращают обход в часы графа.

Трассировка DFS: спуск

Старт из 1: списки смежности $1:[2,4]$, $2:[1,3,5]$, $3:[2,6]$, $4:[1,5]$, $5:[2,4,6]$, $6:[3,5]$, соседи перебираются по возрастанию номеров.

Шаг	Событие	Стек вызовов	Время
1	вход в 1	1	$\mathrm{pre}(1)=1$
2	1 зовёт 2	1, 2	$\mathrm{pre}(2)=2$
3	2 зовёт 3	1, 2, 3	$\mathrm{pre}(3)=3$
4	3 зовёт 6	1, 2, 3, 6	$\mathrm{pre}(6)=4$
5	6 зовёт 5	1, 2, 3, 6, 5	$\mathrm{pre}(5)=5$
6	5 зовёт 4	1, 2, 3, 6, 5, 4	$\mathrm{pre}(4)=6$

Стек вызовов — это текущий путь от старта: колонка стека читается как маршрут $1, 2, 3, 6, 5, 4$.

Трассировка DFS: возвраты

Тупик запускает откат: вызов закрывается, управление получает ближайшая живая развилка.

Шаг	Событие	Стек после шага	Время
7	у 4 нет белых соседей — выход	$1, 2, 3, 6, 5$	$\mathrm{post}(4) = 7$
8	5 досмотрела соседей — выход	$1, 2, 3, 6$	$\mathrm{post}(5) = 8$
9	выход из 6	$1, 2, 3$	$\mathrm{post}(6) = 9$
10	выход из 3	$1, 2$	$\mathrm{post}(3) = 10$
11	2 видит чёрную 5 — выход	1	$\mathrm{post}(2) = 11$
12	1 видит чёрную 4 — выход	$[]$	$\mathrm{post}(1) = 12$

Порядок обхода: $1, 2, 3, 6, 5, 4$ — глубина вместо ширины.

Рёбра $2 — 5$ и $1 — 4$ — это контакты с серыми предками на входах в 5 и 4.

Какие рёбра нашёл обход?

Пример (Два сорта рёбер)

Рёбра дерева — пять спусков из таблицы: $1 — 2$, $2 — 3$, $3 — 6$, $6 — 5$, $5 — 4$.

Обратные рёбра — два контакта с серыми предками: $2 — 5$ и $1 — 4$.

Родитель вершины 5 — это 6, так что серый сосед 2 — предок выше родителя, и ребро $2 — 5$ обратное.

У вершины 4 родитель 5, и серый сосед 1 снова предок выше родителя — ребро $1 — 4$ обратное.

В неориентированном графе всякое недревесное ребро — обратное, и в каждом цикле такое ребро есть.

Цвета обхода: белый, серый, чёрный

Трассировка различала вершины по судьбе в стеке — закрепим различие тремя цветами.

Определение 1 (Цвета вершин)

Белый — вершина ещё не обнаружена. **Серый** — вершина обнаружена, но её вызов ещё не закрыт. **Чёрный** — вершина закрыта: обход вышел из неё и не вернётся.

Серые вершины — это в точности стек вызовов: путь от текущей вершины до корня, и проверка серости соседа — проверка, лежит ли он на текущем пути.

В коде цвета сжимаются в булев флаг `visited`: для BFS безболезненно, но в ориентированном графе теряется различие между обратным ребром и ребром в закрытую вершину.

Что означает ребро в серую вершину?

Теорема 3 (Обратные рёбра и циклы)

DFS находит ребро к серой вершине тогда и только тогда, когда в графе есть цикл.

Доказательство

Докажем эквивалентность.

Серое ребро даёт цикл. Серые вершины — это стек вызовов: v — предок u, и деревянный путь от v до u вместе с ребром $u - v$ образует цикл.

Цикл даёт серое ребро. Пусть v — первая обнаруженная вершина цикла C, остальные при этом белы. При обходе цикла от v каждая следующая вершина либо бела и вызывается из предыдущей, либо уже обнаружена при живом вызове v: весь цикл — потомки v, включая предшественника u вершины v. Вызов u закрывается раньше вызова v, поэтому на рёбрах u вершина v ещё серая: ребро $u - v$ обратное.

Где обход увидел циклы?

Пример (Циклы из трассировки)

Ребро $2 — 5$ замыкает цикл $2 — 3 — 6 — 5 — 2$.

Ребро $1 — 4$ замыкает цикл $1 — 2 — 3 — 6 — 5 — 4 — 1$.

Оба контура собраны без повторных заходов: обход прошёл граф ровно один раз.

Связный граф без циклов — это дерево, так что проверка ацикличности и построение остова — один и тот же обход.

Для леса обходы запускаются, пока остаются белые вершины.

Вопросы для самопроверки — Обходы

1. Почему вершина помечается при входе в очередь, а не при выходе из неё?
2. Как по таблице трассировки DFS узнать обратное ребро?

Времена pre и post

Для каждой вершины интервал $[\mathrm{pre}(u), \mathrm{post}(u)]$ — время её жизни в обходе.

Теорема 4 (Вложенность интервалов)

Для любых вершин u, v интервалы $[\mathrm{pre}(u), \mathrm{post}(u)]$ и $[\mathrm{pre}(v), \mathrm{post}(v)]$ либо не пересекаются, либо один вложен в другой.

Доказательство

Пусть для определённости $\mathrm{pre}(u) < \mathrm{pre}(v)$.

Случай $\mathrm{pre}(v) > \mathrm{post}(u)$. Вызов u закрылся до обнаружения v, интервалы не пересекаются.

Случай $\mathrm{pre}(v) < \mathrm{post}(u)$. v открыта при живом вызове u, значит v — потомок u в дереве обхода. Все потомки закрываются раньше предка, поэтому $\mathrm{post}(v) < \mathrm{post}(u)$: интервал v вложен в интервал u.

Интервалы трассировки

Пример (Интервалы прогона)

В прогоне решётки вершина 1 живёт в интервале $[1, 12]$, накрывающем всех.

Цепь вложенности: $[2, 11]$ у 2, внутри $[3, 10]$ у 3, внутри $[4, 9]$ у 6, затем $[5, 8]$ у 5 и $[6, 7]$ у 4.

Обратным ребро $2 — 5$ стало на входе в 5: сосед 2 был тогда серым, то есть лежал на текущем пути от корня.

Вложенность интервалов подтверждает то же: $[5, 8]$ лежит внутри $[2, 11]$, значит 5 — потомок 2, а ребро ведёт вверх к предку.

Вложенность — это карта рекурсии: кто в чьей ветке стека находился.

На ней держатся поиск циклов, мостов и топологическая сортировка.

Глубина рекурсии

DFS живёт в стеке вызовов, и стек не бесконечен.

Пример

На графе-пути в 10^5 вершин наивная рекурсия падает раньше, чем посетит все вершины.

Лечение — явный стек: тот же порядок обхода без ограничения глубины.

Очереди BFS глубина не грозит: память $O(V)$ при любом графе.

Выбор обхода — ещё и инженерное решение.

BFS против DFS

Задача	BFS	DFS	Почему различие
Кратчайшие пути	да	нет	очередь упорядочена по расстоянию
Циклы	ребро вне дерева	ребро в серую вершину	критерий живёт в орграфе
Топологический порядок	алгоритм Кана	обратный postorder	выход позже достижимых
Компоненты	да	да	любой обход собирает компоненту
Память	$O(V)$, это слой	$O(V)$, это путь	звезда длинна для очереди, путь для стека

Сложность обоих обходов $O(V + E)$, информация разная: BFS отвечает «как далеко», DFS — «как устроено».

Топологическая сортировка

Ориентированный граф без циклов называется **DAG** (*directed acyclic graph*).

- Зависимости сборки: модуль a собирается раньше модуля b.
- Пререквизиты курсов.
- Порядок одевания: носки раньше ботинок.

Определение 2 (Топологический порядок)

Нумерация вершин DAG, при которой каждое ребро $u \to v$ идёт от меньшего номера к большему.

Топологическая сортировка — линейное продолжение частичного порядка, и алгоритм работает за $O(V + E)$.

Алгоритм Кана

Первый способ извлекает вершины без входящих стрелок: исток не зависит ни от кого, его можно выводить сразу.

1. Посчитать входящие степени всех вершин.
2. Поставить в очередь вершины с нулевой входящей степенью — истоки.
3. Пока очередь не пуста: извлечь исток, вывести его, уменьшить степени его соседей — вершина с обнулённой степенью встаёт в очередь.

Пример (Прогон на зависимостях)

Рёбра: core → utils, utils → app, utils → tests, входящие степени: у core ноль, у остальных по единице.

Очередь [core]: извлекаем core, степень utils обнуляется. Извлекаем utils: обнуляются app и tests, вывод — core, utils, app, tests.

Топологический порядок через DFS

Второй способ — полный DFS и выписывание вершин по убыванию времени выхода.

Теорема 5 (Порядок по времени выхода)

В DAG список вершин по убыванию post топологичен: для каждого ребра $u \to v$ вершина u стоит в списке раньше v.

Доказательство

Докажем для каждого ребра $u \to v$ неравенство $\mathrm{post}(u) > \mathrm{post}(v)$.

Случай v белая. DFS переходит из u в v, и потомок закрывается раньше предка.

Случай v серая. Ребро в предка замыкает цикл, которых в DAG не бывает.

Случай v чёрная. v закрылась ещё до входа в u: $\mathrm{post}(v) < \mathrm{pre}(u) < \mathrm{post}(u)$.

Трассировка на графе зависимостей

Пример (Обратный postorder на DAG)

Граф: core $\to$ utils, utils $\to$ app, utils $\to$ tests, соседей перебираем по алфавиту.

Единственный исток — core: входы даёт цепочка $\mathrm{pre}(\mathrm{core}) = 1$, $\mathrm{pre}(\mathrm{utils}) = 2$, $\mathrm{pre}(\mathrm{app}) = 3$, $\mathrm{pre}(\mathrm{tests}) = 5$.

Выходы: $\mathrm{post}(\mathrm{app}) = 4$, $\mathrm{post}(\mathrm{tests}) = 6$, $\mathrm{post}(\mathrm{utils}) = 7$, $\mathrm{post}(\mathrm{core}) = 8$.

По убыванию выходов: core, utils, tests, app — каждая стрелка смотрит вперёд.

Порядок отличается от результата Кана перестановкой app и tests — оба верны, пути между этими вершинами нет.

На графе с циклом список не будет топологическим. Сигналы беды: ребро в серую вершину у DFS, у Кана — выведено меньше вершин, чем n: оставшиеся образуют цикл, и сборка невозможна.

Зависимости в инженерии

Пример (Сборка и пакетные менеджеры)

Сборщик `make` читает граф целей: цель — вершина, зависимость — стрелка.

`npm` и `cargo` разрешают дерево пакетов так же: сначала зависимости, потом зависящие от них.

Цикл сборщик не пропустит: `make` сообщает о circular dependency, менеджеры пакетов отказывают в установке.

Разрешение зависимостей — это топологическая сортировка: ни один шаг не выполняется раньше своих предпосылок.

Топологический порядок фиксирует лишь частичный порядок: независимые цели `make -j` запускает параллельно.

Обходы в коде: BFS

```rust
use std::collections::VecDeque;
fn bfs(adj: &[Vec<usize>], start: usize) {
    let mut parent = vec![None; adj.len()];
    let mut seen = vec![false; adj.len()];
    let mut order = vec![];
    let mut q: VecDeque<_> = [start].into();
    seen[start] = true;
    while let Some(u) = q.pop_front() {
        order.push(u);
        for &v in &adj[u] {
            // пометка при входе в очередь
            if !seen[v] { seen[v] = true; parent[v] = Some(u); q.push_back(v); }
        }
    }
    println!("order {order:?}\nparent {parent:?}");
}
```

Сложность не зависит от формы графа: каждая вершина входит в очередь один раз, каждое ребро просматривается дважды.

Обходы в коде: DFS

Рекурсивная версия повторяет таблицу трассировки: стек вызовов виден в сигнатуре.

```
fn dfs(u: usize, adj: &[Vec<usize>], seen: &mut [bool],
    parent: &mut [Option<usize>], order: &mut Vec<usize>) {
    seen[u] = true;
    order.push(u);
    for &v in &adj[u] {
        if !seen[v] {
            parent[v] = Some(u);
            dfs(v, adj, seen, parent, order);
        }
    }
}
```

Обходы в коде: вывод

Пример (Вывод на решётке)

BFS печатает порядок $1, 2, 4, 3, 5, 6$ и родителей $[-, 1, 2, 1, 2, 3]$ — запись с номером v отвечает родителю вершины v.

DFS печатает $1, 2, 3, 6, 5, 4$ — в точности порядок спуска из таблицы трассировки.

Флаг `seen` включается при постановке в очередь — тот самый инвариант трассировки.

Без него вершина встанет в очередь по разу на каждое входящее ребро.

Вопросы для самопроверки — Топология и код

1. Что означает встреча с серой вершиной в ориентированном графе?
2. Почему полный DFS для леса не портит линейность обхода?

Обходы и код

- Очередь BFS — `deque` в Python или `VecDeque` в Rust.
- Рекурсия DFS — стек вызовов, при нехватке глубины переписывается на явный стек.
- Сборщики `make` и `ninja` строят граф зависимостей и запускают цели в топологическом порядке.
- Сборщик мусора обходит граф живых объектов — пометкой (mark-and-sweep).

> «Пометить вершину» в коде — это флаг `visited`: без него обход зациклится на первом же цикле.

Деревья

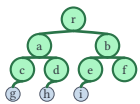
> *Родственные связи всех существ одного класса иногда изображали в виде великого дерева. Я верю, что эта метафора во многом соответствует истине.*
>
> ***— Чарльз Дарвин***

Дерево и лес

Определение 3

Дерево — связный граф без циклов.

Лес — граф без циклов.



Пример

Цепочка $1 - 2 - 3 - 4 - 5$ — дерево: 5 вершин и $4 = 5 - 1$ ребро, а два треугольника — уже не лес: в каждом цикл.

Сколько рёбер у дерева?

Теорема 6 (Число рёбер)

Дерево на n вершинах имеет ровно $n - 1$ ребро, а при $n \geq 2$ — хотя бы два листа.

Доказательство

Докажем индукцией по n, предварительно найдя лист.

Листья. В самом длинном простом пути концы имеют ровно одного соседа: лишний сосед удлинил бы путь или создал бы цикл, и при $n \geq 2$ листьев хотя бы два.

Число рёбер. Шаг индукции: при $n = 1$ рёбер нет, а отделив лист, получаем дерево на $n - 1$ вершине с $n - 2$ рёбрами, и с ребром листа выходит ровно $n - 1$.

Дерево — минимальный связный каркас: убрать ребро — разрыв, добавить — цикл.

Пять характеризаций

Теорема 7 (Эквивалентные характеризации дерева)

Для графа с n вершинами эквивалентны:

1. связный и ациклический.
2. связный и имеет $n - 1$ ребро.
3. ациклический и имеет $n - 1$ ребро.
4. между любой парой вершин — единственный простой путь.
5. связный, и каждое ребро — мост.

Пример (Проверка на пути)

Путь $1 - 2 - 3 - 4 - 5$: связен, циклов нет, $4 = 5 - 1$ ребро, между 1 и 5 ровно один простой путь, ребро $2 - 3$ — мост. Контрпроверка: после добавления ребра $1 - 5$ появится цикл, и характеризации 2 и 3 нарушатся.

Цепочка эквивалентностей

Достаточно замкнуть круг из пяти импликаций.

Доказательство

(1) ⇒ (2). Это теорема о числе рёбер: у дерева ровно $n - 1$ ребро.

(2) ⇒ (3). Срезая рёбра циклов, получили бы дерево с менее чем $n - 1$ ребром: противоречие.

(3) ⇒ (4). Компонента — дерево, всего $n - c$ ребра, и $n - 1$ означает $c = 1$. Два разных пути между одной парой дают цикл в симметрической разности — противоречие.

(4) ⇒ (5). Пути между парами существуют, значит граф связен, а удаление ребра $u - v$ губит единственный путь: каждое ребро — мост.

(5) ⇒ (1). Ребро цикла мостом не бывает, значит циклов нет.

Деревья вокруг

Пример (Деревья вокруг)

- Файловая система: каталоги и файлы, корень — `/`, у каждого файла ровно один родительский каталог.
- DOM страницы: теги и вложенность, у элемента ровно один непосредственный родитель.
- Наследование классов в языках с одиночным наследованием: базовый класс — корень.
- Родословная одного рода: у каждого человека ровно один отец и ровно одна мать.

Каждый раз вопрос один: у каждого объекта ровно один «родитель», корень — начало.

Родитель всегда стоит уровнем ближе к корню, поэтому циклы исключены, а достижимость от корня даёт связность — требования дерева в чистом виде.

Остовное дерево

У каркаса из рёбер к новым вершинам есть имя.

Определение 4

Остовное дерево (*spanning tree*) графа — подграф из всех вершин, являющийся деревом.

Теорема 8 (Существование)

Любой связный граф имеет остовное дерево.

Доказательство

Обойдём граф, оставив рёбра к белым вершинам: каждое добавляло новую вершину, циклов нет, и по первой характеризации получилось дерево.

Сколько остовных деревьев?

Теорема 9 (Формула Кэли)

Полный граф K_n имеет ровно n^{n-2} остовных деревьев.

Пример

K_4: $4^2 = 16$ остовных деревьев.

K_{10}: уже 10^8 — перебор безнадёжен, нужен алгоритм.

Кэли вывел формулу в 1889 году.

Крускал и Прим

Рёбрам приписаны веса — ищем остов минимального суммарного веса.

- **Крускал**: сортируем рёбра, добавляем легчайшее, не создающее цикла.
- **Прим**: растим дерево из одной вершины, добавляя легчайшее ребро наружу.

Оба алгоритма жадные, и оба оптимальны.

Жадность оптимальна не везде: здесь работает обменный аргумент — выбранное ребро вписывается в минимальный остов заменой ребра не меньшего веса, и стоимость остова не растёт.

Проверка «не создаёт ли ребро цикла» в Крускале — это система непересекающихся множеств: компоненты леса сливаются по мере добавления рёбер.

Остов минимального веса на числах

Пять городов соединяют семь дорог.

Шаг	Ребро	Цена	Судьба
1	$2-3$	1	в остов
2	$1-2$	2	в остов
3	$1-3$	3	цикл $1-2-3$
4	$2-4$	4	в остов
5	$3-4$	5	цикл $2-3-4$
6	$4-5$	6	в остов
7	$3-5$	7	остов уже собран

Остов $1-2$, $2-3$, $2-4$, $4-5$ стоит $1+2+4+6=13$.

Прим из вершины 1 находит то же дерево: те же четыре ребра входят в него в другом порядке.

Проверьте себя

Вопросы для самопроверки

1. Почему BFS находит кратчайшие пути, а DFS — нет?
2. Что означает вложенность интервалов pre и post? Что она говорит о рекурсии?
3. Докажите, что в дереве с $n \geq 2$ есть хотя бы два листа.
4. Объясните, почему граф связен тогда и только тогда, когда в нём есть остовное дерево.
5. В каком порядке сборщик обработает цели, если зависимости образуют цикл?

Что мы поняли?

Обходы превратили граф из рисунка в структуру с порядком посещения — и на обоих взглядах выросли рабочие алгоритмы.

- Обходы идут за $O(V + E)$: BFS — очередью и слоями, DFS — стеком и временами pre и post.
- BFS даёт кратчайшие пути в невзвешенном графе, DFS — циклы и топологический порядок, алгоритм Кана и обратный postorder — за линейное время.
- Дерево — связный ациклический граф: $n - 1$ ребро, единственный путь, все рёбра — мосты.

- m09 — «Обходы графа», «Топологическая сортировка», «Деревья», «Минимальные остовные деревья».